

Part 5 - Frontend, Deployment and Submission

What the user sees, how to host it, what to document and what to submit.

How to read this pack

Blue boxes are explicit case-study requirements recovered from the source explanation. Green boxes are implementation recommendations or inferences. This distinction helps you explain what was mandatory and what you chose as a developer.

Language style: simple Hinglish, interview-focused, and implementation-oriented. The goal is to help you build the project and confidently explain why each part exists.

1. Frontend Goal

Case-study requirement

Frontend should be React-based, responsive and admin-style. Submission expects README, live URLs, test credentials, API documentation, architecture explanation and known limitations.

Frontend ka purpose hai business users ko simple screens dena jahan wo customers, products, stock and challans manage kar sake. UI beautiful ho sakti hai, but sabse important hai usable and complete workflow.

2. Suggested Layout

Top bar: Mini ERP + CRM | logged-in user | logout
Sidebar: Dashboard, Customers, Products, Inventory, Challans, Users
Main area: tables, forms, detail views and actions

3. Core Screens

Screen	What it should show
Login	Email, password, login button, error state
Dashboard	Total customers, total products, low-stock count, today's challans, recent challans
Customers	Search/filter table, add/edit customer, customer detail and notes
Products	Product table, SKU/category/price/stock, low-stock badge
Inventory	Stock movement log with IN/OUT, reason, created by, timestamp
Challans	List, create draft, add items, confirm/cancel, detail page
Users/Admin	Optional or admin-only user management if time allows

Recommended implementation

Build dashboard cards only after core modules work. A polished dashboard helps demo, but working challan and stock logic are more important.

4. Frontend State and API Handling

- Store JWT after login and send it in Authorization header.
- Redirect unauthenticated users to login.
- Hide menu items based on role for better UX.
- Still depend on backend for real permission enforcement.
- Show loading, empty and error states for tables.
- Show readable business errors, especially insufficient stock.

5. Deployment Options

Case-study requirement

Deployment is expected. AWS is preferred but not mandatory. Frontend can use Vercel, Netlify, Render Static Site or similar. Backend can use Render, Railway, Fly.io or similar. Database can use Supabase, Neon, Render Postgres or similar.

48-hour assignment ke liye complex AWS setup mein time waste mat karo unless tum already comfortable ho. Simple hosted stack acceptable hai as long as app live hai and reviewer use kar paaye.

```
Recommended hosting path:  
React frontend -> Vercel  
Node/Express backend -> Render  
PostgreSQL database -> Neon or Supabase
```

6. Environment Variables

Case-study requirement

Environment-variable management should be documented.

Secrets ko GitHub mein commit nahi karna. DATABASE_URL, JWT_SECRET, API URLs, ports etc. .env mein rakhna aur .env ko .gitignore mein add karna.

```
Backend .env example:  
DATABASE_URL=postgresql://...  
JWT_SECRET=change_this_secret  
PORT=5000  
CORS_ORIGIN=https://your-frontend.vercel.app  
  
Frontend .env example:  
VITE_API_BASE_URL=https://your-backend.onrender.com
```

7. GitHub Commit Strategy

Case-study requirement

Proper commits and README are expected.

One giant commit 'final project' weak impression deta hai. Multiple meaningful commits show planning and discipline.

```
Suggested commits:  
Initial project setup  
Setup database schema  
Implement authentication and roles  
Add customer CRM module  
Add product inventory module  
Implement challan workflow  
Add stock validation and movement log  
Build React dashboard and forms  
Add API documentation  
Deploy application and update README
```

8. README Checklist

- Project overview: what problem the portal solves.
- Tech stack: frontend, backend, database and hosting.
- Architecture: React -> REST API -> Node/Express -> SQL database.
- Features: auth, roles, CRM, products, inventory, challans.
- Database design: tables and relationships.
- API endpoints: method, path, purpose and status codes.
- Environment variables: list and sample values without secrets.
- Local setup: install, migrate/seed, run backend, run frontend.
- Deployment URLs: frontend and backend.

- Test credentials: Admin, Sales, Warehouse, Accounts.
- Assumptions and known limitations.

9. Submission Package

Item	Why reviewer needs it
GitHub repo	To inspect code quality and commits
Live frontend URL	To test app quickly
Backend/API URL	To verify APIs if needed
Test credentials	To login as each role
README	To understand setup, architecture and assumptions
API docs	To see backend contract
Known limitations	To show honesty and scope control